

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

The recent proliferation of optimization algorithms for deep learning—among them Schedule-Free optimizers, AdEMAMix, MARS, and Lion—has been driven in large part by a shared impulse to reconfigure the conventional momentum mechanism. Along an ostensibly independent trajectory, theoretical work has secured provable acceleration of stochastic gradient descent (SGD) within the noise-dominated regime, an achievement predicated on decoupling the momentum coefficient from the weight apportioned to the instantaneous gradient. The present paper articulates the explicit correspondences that bind these two hitherto disjoint literatures. We buttress our theoretical characterization with preliminary experiments on a 150m language-modeling task, whereupon we observe that AdEMAMix—the method whose structure most faithfully recapitulates accelerated variants of stochastic gradient descent—attains markedly superior performance. Capitalizing on this observation, we advance a modification of AdEMAMix, designated Simplified-AdEMAMix, which preserves the performance of its progenitor across both large- and small-batch regimes while obviating the necessity of maintaining two disparate momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

The recent past has witnessed a marked proliferation of optimization algorithms devised for deep learning, prominent instances of which include Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Notwithstanding the heterogeneous motivations under which these optimizers were conceived, they are united by a single structural commonality: each intervenes upon the momentum scheme that governs the optimization trajectory.

A separate and largely self-contained body of theoretical inquiry has concerned itself with the acceleration of gradient descent under conditions of stochastic noise. While the classical momentum methods—heavy-ball and Nesterov momentum among them—suffice to accelerate deterministic gradient descent, and do so conspicuously for quadratic objectives, they confer no such acceleration upon SGD (Jain et al., 2018; Liu & Belkin, 2020). It is precisely this deficiency that has precipitated the formulation of alternative momentum schemes explicitly engineered to recover acceleration in the presence of noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Tellingly, the entirety of the accelerated SGD methods hitherto proposed admits a unifying interpretation: each effects a decoupling of the momentum coefficient from the weight that the optimizer update assigns to the current gradient.

Our principal contribution consists in rendering explicit the direct correspondence between the conceptual apparatus developed within these two research directions. Concretely, we establish that Schedule-Free SGD is mathematically equivalent to the composition of accelerated SGD with subsequent weight averaging. We demonstrate, moreover, that optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix are amenable to interpretation as conjunctions of preconditioning techniques with accelerated SGD procedures. Although fragments of these correspondences have received prior acknowledgment in the literature (Defazio, 2021), the relationship between these recently proposed optimizers and accelerated SGD has not, to the best of our knowledge, previously been placed on a formal footing.

In order to corroborate our theoretical characterization, we conduct experiments employing a 150m decoder-only transformer model, trained on 15b tokens under a small batch size of 32k tokens—a configuration deliberately chosen to guarantee that training unfolds within a noise-dominated regime. In consonance with our theoretical predictions, the performance of Schedule-Free AdamW proves closely commensurate with that of accelerated SGD-based AdamW (Algorithm 3). We observe, in addition, that accelerated methods furnish a modest performance advantage

at small batch sizes. We nevertheless demonstrate that these benefits attenuate at sufficiently large batch sizes, an outcome wholly consonant with the theoretical connections to accelerated SGD.

The principal contributions of this work may be enumerated as follows:

1. We render precise the theoretical correspondences that obtain between accelerated SGD and a range of recently proposed optimizers, Schedule-Free SGD and AdEMAMix foremost among them.
2. We furnish empirical corroboration by way of experiments on a 150-million-parameter decoder-only transformer, juxtaposing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS; our findings indicate that AdEMAMix, whose structure most faithfully aligns with the accelerated SGD variants, attains performance superior to that of its competitors.
3. As anticipated from its equivalence to accelerated SGD, the performance advantages conferred by these methods dissipate at large batch sizes relative to Adam; we demonstrate, notably, that Adam endowed with momentum scheduling suffices to match the performance of AdEMAMix.
4. We observe that, at high batch sizes, Schedule-Free AdamW is substantially outperformed by AdamW with cosine decay, a deficit we ascribe to the intrinsic coupling of the momentum and weight-averaging coefficients that characterizes Schedule-Free optimizers.
5. We introduce a modification of AdEMAMix, designated Simplified-AdEMAMix, which preserves the performance of AdEMAMix across both large- and small-batch regimes while dispensing with the need for two distinct momentum terms.

2 RELATED WORK

We survey the extant literature bearing directly upon our work, encompassing both accelerated SGD variants and the broader family of optimization algorithms to which they are germane.

Jain et al. (2018) inaugurated an accelerated SGD variant that exhibited improved convergence rates on the least-squares problem. Building upon this, Kidambi et al. (2018) distilled the update rule into a simpler form and formally certified that, in this particular case, momentum affords no acceleration. Subsequent contributions (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these findings to general convex and strongly convex objectives under an assortment of theoretical assumptions.

Over the intervening years, a number of optimizers have been proposed that bear a discernible resemblance to the accelerated SGD variants adumbrated above. Lucas et al. (2019) advanced a method that assembles the final update from a weighted sum of multiple momentum terms, each governed by its own distinct coefficient. Ma & Yarats (2019) devised an optimizer expressly inspired by the theoretical framework laid down in Jain et al. (2018). More recently, Chen et al. (2023) introduced an optimizer uncovered through a genetic search procedure which, in the manner of its accelerated SGD antecedents, apportions dissimilar weights to the gradient and to the momentum coefficient at the update step. Pagliardini et al. (2024), for their part, proposed a method that amalgamates two distinct momentum scales within the final update.

3 BACKGROUND

3.1 MOMENTUM

Momentum constitutes a well-established device for expediting the convergence of gradient descent in deterministic settings. The momentum update for the weights w_t , parameterized by a momentum coefficient β , assumes the form:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a technique in widespread use throughout stochastic optimization, its purpose being the suppression of noise in the iterates. Rather than returning the terminal iterate w_T , one computes a weighted average $\bar{w}_T$ over the iterates, the associated weights being denoted γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, every instance of weight averaging employs coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

In the present section, we set forth a generalized framework capacious enough to subsume a broad class of accelerated SGD methods:

$$m_t = \beta_{a,t}m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t}m_t - \alpha_{a,t}g_t \quad (1)$$

wherein $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ denote scalar coefficients that may in general be time-dependent, and g_t designates the stochastic gradient evaluated at w_t . The subscript ‘a’ serves throughout to demarcate coefficients that conform to this particular accelerated SGD formulation.

We remark, at the outset, that the choice $\alpha_{a,t} = 0$ recovers standard SGD with momentum. Furthermore, as has been observed in prior work, a substantial proportion of the accelerated SGD algorithms advanced in the literature—among them those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—fall squarely within the ambit of this framework. A rigorous demonstration of this equivalence is deferred to Appendix B.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section, we establish on theoretical grounds the precise correspondences that link existing optimizers—Schedule-Free optimizers and AdEMAMix among them—to accelerated SGD. Proceeding from these insights, we propose a simplified variant of AdEMAMix that operates with but a single momentum term, yet sustains performance commensurate with that of AdEMAMix across both the small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a recently introduced constant-learning-rate optimizer conceived expressly to dispense with the need for scheduling. Adhering to the notation adopted in Defazio et al. (2024), its update equations read:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here y_t designates the current model weights, at which the gradient is evaluated, whereas x_t denotes the weights reserved for evaluation.

We begin by recasting the update in terms of y_t and m_t , the latter being defined according to:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Upon further simplification of m_{t+1} , we arrive at:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Consequently, m_t obeys the momentum update of Equation (1) with $\beta_{a,t} = 1 - c_t$. Having thus characterized m_t , we now turn to an examination of the update governing y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Accordingly, y_t obeys the weight update of Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. It follows that y_t in Schedule-Free SGD conforms exactly to the accelerated SGD framework. It is x_t , however, that serves for evaluation within Schedule-Free SGD, and we accordingly proceed to an analysis of its dynamics:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Hence x_t constitutes a weighted average of y_t . A recursive expansion of x_t confirms that, whenever c_t is held constant, it amounts to an exponential average of y_t . This establishes that Schedule-Free SGD is properly to be understood as accelerated SGD succeeded by weight averaging.

The efficacy of Schedule-Free SGD is attributable to two constituent components:

1. performance improved relative to standard SGD with momentum, a consequence of its equivalence to accelerated SGD;
2. the capacity to operate at a constant learning rate absent any scheduling, a capacity conferred by weight averaging—specifically, tailed weight averaging; see Section 4.1.3.

We further note two advantages peculiar to Schedule-Free SGD/Adam:

- it exacts no additional memory on account of weight averaging;
- it obviates the introduction of an explicit weight-averaging coefficient as a hyperparameter.

In Section 5.1, however, we demonstrate that this very coupling of the momentum and weight-averaging coefficients scales poorly to large batch sizes.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), when $\beta = 0$ Schedule-Free SGD collapses to standard SGD supplemented by weight averaging; since $c_t = 1/t$ in this case, averaging is applied from the very outset.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), when $\beta = 1$ Schedule-Free SGD collapses to standard momentum SGD, wherein the momentum coefficient $\beta_{a,t}$ scales as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For $\beta = 0.9$, the default configuration of Schedule-Free SGD, the following observations obtain:

- inasmuch as c_t scales as $1/t$, the momentum grows as $1 - 1/t$;
- the ratio of the weight borne by the current gradient to that borne by momentum remains fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$;
- weight averaging is applied, to a close approximation, over the most recent 10% of the iterates.

4.2 LION

The update rule that defines Lion (Chen et al., 2023) is given by:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) thus admits a direct interpretation as an accelerated SGD method succeeded by a coordinate-wise sign operation.

4.3 MARS

In this section, we demonstrate that the practical instantiation of the recently proposed optimizer MARS (Yuan et al., 2024), denoted MARS-Approx, conforms to the accelerated SGD framework once augmented by a preconditioning step. The update equations, setting aside bias correction and clipping, read:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

wherein m_t and v_t denote, respectively, the first- and second-order momentum terms, and x_t designates the model parameters. Recasting the update by means of the substitution $\hat{m}_t = m_t - \gamma g_t$, we obtain:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This formulation makes manifest that the momentum update conforms to the general accelerated SGD framework. It bears emphasis, nonetheless, that MARS avails itself of a preconditioning approach distinct from that of AdamW. Its empirical performance is subjected to further analysis in Section 5.

4.4 ADEMAMIX

The recently proposed optimizer AdEMAMix (Pagliardini et al., 2024) exhibits pronounced structural affinities with accelerated SGD-based AdamW. In contradistinction to accelerated SGD, however, which forms a linear combination of the current gradient and the momentum term, AdEMAMix maintains two distinct momentum terms endowed with differing coefficients and computes a linear combination thereof. The algorithm is stated formally in Algorithm 1.

For the sake of analytical tractability, we consider the variant of AdEMAMix specified by $\beta_1 = 0$. As demonstrated in Pagliardini et al. (2024), this simplified version attains performance all but indistinguishable from that of the full version at small batch sizes—a finding our experiments in Section 5 corroborate. Under $\beta_1 = 0$, AdEMAMix falls into alignment with the general accelerated SGD framework (Equation (1)). We show, moreover, that the schedules prescribed for β_3 , the momentum coefficient, and for α , which regulates the relative weight apportioned to the current gradient, adhere closely to the theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, the attainment of acceleration in stochastic settings is contingent upon a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). The AdEMAMix optimizer approximates this scheme by scaling up β_3 commensurately.

Note, additionally, that within accelerated SGD schemes momentum is maintained in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas in Algorithm 1 the momentum update instead proceeds according to:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients to m_t in accelerated SGD grows in proportion to t ; by the same token, the coefficient α in AdEMAMix likewise scales in proportion to t . It is precisely to these structural affinities that we attribute the improved empirical performance AdEMAMix displays relative to competing optimizers, as evidenced in Figure 1.

At large batch sizes, however, AdEMAMix suffers a deterioration in performance under $\beta_1 = 0.0$, as reported in Pagliardini et al. (2024). Gupta et al. (2023) intimates that, for large batch sizes, the weight borne by the current gradient in the update ought to diminish; AdEMAMix, by contrast, holds this weight fixed at unity, a stipulation that plausibly underlies its degraded performance in the large-batch setting.

In the section that follows, we introduce a simplified variant of AdEMAMix that reinstates a weight on the current gradient, dispenses with the scheduling of α , and retains but a single momentum term. We validate empirically that this simplified version performs on a par with AdEMAMix across both the small- and large-batch setups.

4.5 SIMPLIFIED ADEMAMIX

Proceeding from the insights adduced above, we propose a simplified optimizer that dispenses at once with the maintenance of two separate momentum terms and with the scheduling of α . The optimizer is presented formally in Algorithm 2, wherein we adopt theory-style momentum in preference to the exponential-moving-average (EMA) formulation, and in the final update assign a fixed weight α to the gradient. We remark that the choice $\alpha = 0$ recovers the standard Adam optimizer, subject to suitable transformations of η and β_1 . In Section 5 we demonstrate that this simplified variant matches the performance of AdEMAMix across both the small- and large-batch regimes.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

5 EXPERIMENTS

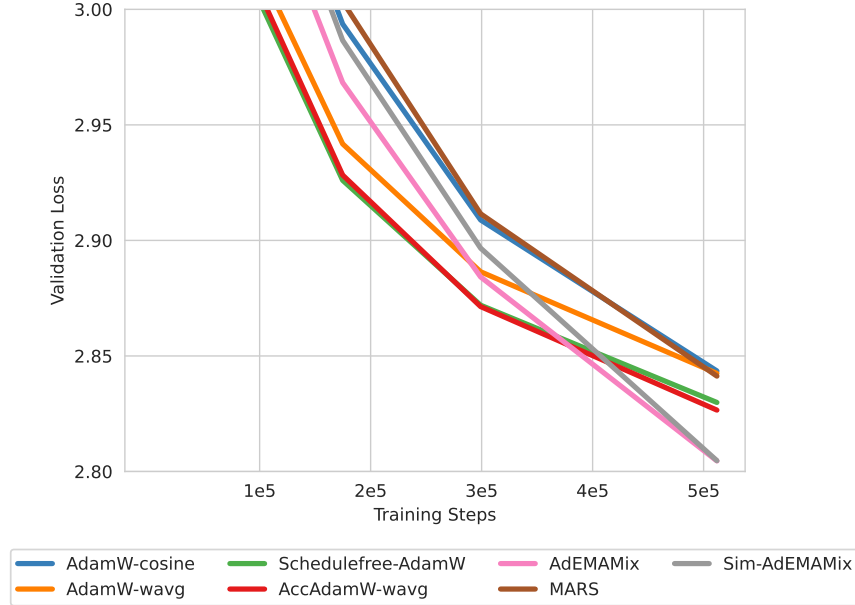


Figure 1: A juxtaposition of the best runs of the several optimizers enumerated in Section 5, evaluated on a language-modeling task with a decoder-only 150m transformer model. We find that AdEMAMix and simplified-AdEMAMix prove the most performant, an outcome we attribute to the exactness of their correspondence with the accelerated SGD variants.

In this section, we present the experiments conducted on a 150-million-parameter decoder-only transformer model for a language-modeling task drawing upon the C4 dataset. The model is trained with a sequence length of 1024 and a batch size of 32, over 15 billion tokens ($\approx 5 \times$ Chinchilla)—a token budget calibrated so as to ensure that training operates within a noise-dominated regime.

The optimization algorithms placed in comparison are the following:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)

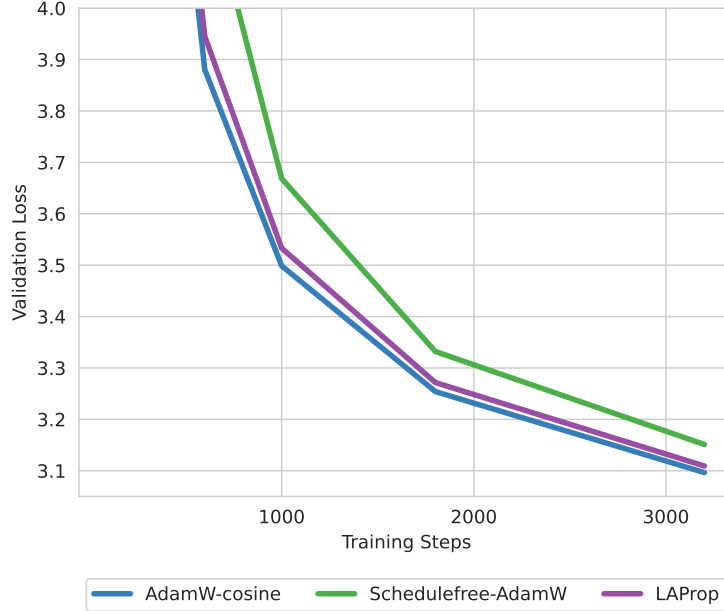


Figure 2: A juxtaposition of the best runs of AdamW with cosine decay, Schedule-Free AdamW, and LARProp at the higher batch size. The attendant experimental details are set out in Section 5.1.

5. MARS

6. AdEMAMix

7. Simplified-AdEMAMix

The particulars of the hyperparameter sweeps conducted for these algorithms are furnished in Section A.

As illustrated in Figure 1, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform comparably, a concordance that lends support to our theoretical claims; both, furthermore, outperform AdamW with cosine decay as well as AdamW with tailed weight averaging. AdEMAMix and Simplified-AdEMAMix, for their part, surpass every competing method—a superiority we conjecture to be a consequence of their alignment with the accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

Whereas the foregoing experiments were confined to the small-batch regime—that is, to training under noisy gradients—we now undertake experiments in the large-batch regime in order to ascertain whether these algorithms generalize with equal efficacy. To this end, we train models with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: As shown in Figure 2, Schedule-Free AdamW is markedly outperformed by AdamW. We ascribe this shortfall to the coupling between the weight-averaging and momentum coefficients: at higher batch sizes the optimal momentum value lies considerably below $1 - 1/t$, and while one might in principle adopt a scaling factor $\approx 1 - r/t$ for some $r \geq 1$, any increase in r contracts the effective weight-averaging window.

A further salient distinction between AdamW and Schedule-Free AdamW resides in the order in which momentum and preconditioning are applied: AdamW applies momentum in advance of preconditioning, whereas Schedule-Free AdamW inverts this ordering, applying preconditioning before momentum and thereby rendering itself algorithmically akin to LARProp (Ziyin et al., 2021). As Figure 2 makes plain, however, the performance of AdamW proves comparable to that of LARProp, which suggests that this distinction is not the principal source of the performance gap.

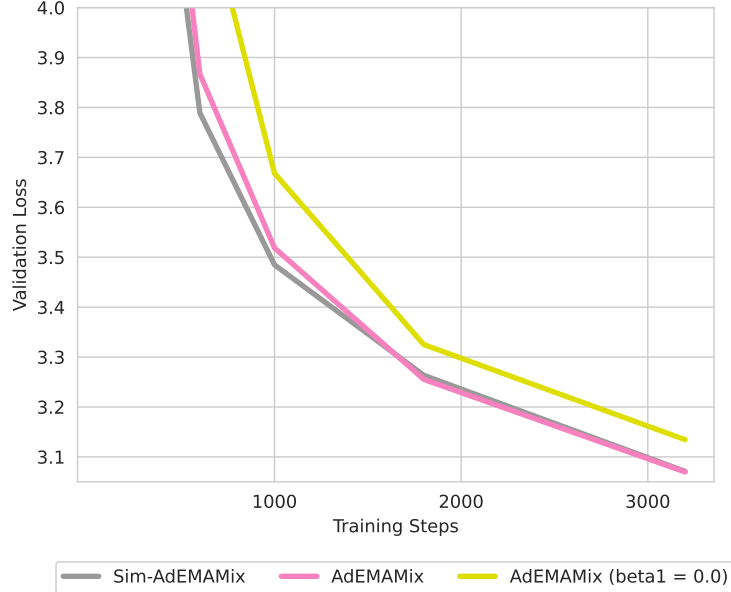


Figure 3: A juxtaposition of the best runs of AdEMAMix (both with and without $\beta_1 = 0.0$) and our simplified AdEMAMix variant across the higher-batch-size experiments. The attendant experimental details are set out in Section 5.1.

AdEMAMix: At large batch sizes, in accordance with the observations previously reported in Pagliardini et al. (2024), Figure 3 reveals that the choice $\beta_1 = 0.0$ in AdEMAMix precipitates a substantial degradation in performance relative to the use of two separate momentum terms. This degradation arises because AdEMAMix holds the weight on the current gradient fixed at unity, whereas the theoretical accelerated SGD variants (Gupta et al., 2023) demand a weight on the current gradient that diminishes as the batch size grows.

As Figure 3 further depicts, our proposed variant, Simplified-AdEMAMix, attains performance equivalent to that of AdEMAMix while obviating the need for two separate momentum terms. Notably, this performance is realized at $\alpha = 0.0$, whereupon Simplified-AdEMAMix reduces to standard Adam endowed with momentum scheduling.

6 CONCLUSION

In this work, we have rendered explicit the connections that bind the accelerated SGD variants to several recently proposed optimizers, Schedule-Free optimizers, AdEMAMix, MARS, and Lion among them. We have furthermore adduced empirical evidence demonstrating that AdEMAMix—the method whose structure aligns most closely with the theoretical accelerated SGD variants—attains superior performance in small-batch training.

Proceeding from this connection, we have introduced Simplified-AdEMAMix, which dispenses with the maintenance of two separate momentum buffers. We have shown empirically that Simplified-AdEMAMix matches the performance of AdEMAMix across both the small- and large-batch regimes while eliminating the supplementary memory overhead that AdEMAMix incurs.

REFERENCES

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, De-*

- ember 10 - 16, 2023, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

We set out below the hyperparameters governing the small-batch experiments.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 proved to be .9 and .999 respectively, coinciding with the default values. We note that at larger batch sizes it is customary to adopt $\beta_2 = .95$; the benefit conferred by a higher β_2 at smaller batch sizes has likewise been documented by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This configuration fared worse than the warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1\text{e-}6, 3.16\text{e-}6, 1\text{e-}5, 3.16\text{e-}5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The hyperparameter sweeps pertaining to the large-batch experiments are furnished below:

1. Schedule-Free AdamW: $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1\text{e-}4, 3.16\text{e-}4, 1\text{e-}3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

The general accelerated SGD form is that of Equation (1). In this section, we establish that the entirety of the methods advanced in the works Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) fall within this form.

B.1 AGNES

The update proposed by Gupta et al. (2023) is given below:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n denotes the stochastic gradient evaluated at x'_n and the final function is evaluated at x_n . The equations above may be recast as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad - \frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(- \frac{v_n}{\rho_{n-1}} \right) + g'_n$$

whence x'_{n+1} obeys an update equation of the form of Equation (1).

B.2 ASGD

The update proposed by Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} denotes the stochastic gradient evaluated at y_{j-1} and the function is evaluated on the tail-averaged x .

The update equations above admit the reformulation:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The update equations above thus conform to the form of Equation (1).

B.3 MASS

The update proposed by Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t denotes the stochastic gradient evaluated at u_t and the function is evaluated at w_t . These equations may be recast as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The update equations above thus conform to the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update proposed by Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations may be recast as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$